المدرسة الوطنية للعلوم التطبيقية
ⵜⵉⵎⵙⵙⵓⵜ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵙⴻⵏⵙⵉⵡⵉⵏ ⵜⵉⵎⵓⵔⵉⵡⵉⵏ
Ecole Nationale des Sciences Appliquées de Fès

جامعة سيدي محمد بن عبد الله
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵙⵉⴷⵉ ⵎⵓⵃⴰⵎⵎⴻⴷ ⴱⵏ ⵄⴱⴷⵍⵍⴰⵀ
Université Sidi Mohamed Ben Abdellah

TP1 :

AWS Academy Cloud Foundations





BELHAJ KHAOULA

N 9

GSCSI- G1

Table des matières

Table des matières 2
Table de figures 3
Atelier 3 : Introduction à Amazon EC2 4
Introduction 4
Tâche 1 : Lancement de votre instance Amazon EC2 6
Tâche 2 : Surveillance de votre instance 8
Tâche 3 : Mise à jour de votre groupe de sécurité et accès au serveur web 10
Tâche 4 : Redimensionnement de votre instance - type d'instance et volume EBS 12
Tâche 5 : Exploration des limites d'EC2 14
Tâche 6 : Test de la protection contre l'arrêt 15
Fin de l'atelier 16
Conclusion 17
Automatisation avec AWS Lambda et EventBridge 18
Introduction : 18
Configuration de la fonction Lambda 19
Tâche 1 : Création d'une fonction Lambda 21
Tâche 2 : Configuration du déclencheur 22
Tâche 3 : Configuration de la fonction Lambda 23
Tâche 4 : Vérification du bon fonctionnement de la fonction Lambda 24
Conclusion 25

Table de figures

Figure 1 console EC2 6
Figure 2 État de l'instance EC2 après le lancement initial. 7
Figure 3 Exemple métriques Amazon CloudWatch 8
Figure 4 Log système confirmant l'exécution réussie du script de post-installation. 9
Figure 5 Obtenir la capture de l'instance 9
Figure 6 Web Server Détails 10
Figure 7 Serveur inaccessible 10
Figure 8 Modification du groupe de sécurité 11
Figure 9 Ajout HTTP 11
Figure 10 Confirmation de la connectivité HTTP après modification des règles de sécurité .. 11
Figure 11 Arrêt de l'instance 12
Figure 12 Passage de t2.micro à t2.small 12
Figure 13 Modification du volume EBS 12
Figure 14 Extension de la capacité de stockage du volume racine. 13
Figure 15 Activation de la protection contre l'arrêt 13
Figure 16 Service Quotas 14
Figure 17 Running On-Demand Standard 14
Figure 18 Démonstration de la sécurité empêchant l'arrêt accidentel du serveur 15
Figure 19 Désactivation et Arrêt final 15
Figure 20 Fin de l'atelier 16
Figure 21 créer la fonction nommée myStopinator 19
Figure 22 Création de la fonction Lambda avec les permissions d'exécution nécessaires 20
Figure 23 EventBridge 21
Figure 24 Console AWS Lambda 23
Figure 25 instance arrêtée 24
Figure 26 Fin de l'atelier 24

Atelier 3 : Introduction à Amazon EC2

Introduction

Le paradigme du Cloud Computing

L'émergence du Cloud Computing, et plus particulièrement du modèle IaaS (*Infrastructure as a Service*), a radicalement transformé la gestion des ressources informatiques en entreprise. Amazon Web Services (AWS), via son service phare Amazon Elastic Compute Cloud (Amazon EC2), permet désormais de s'affranchir des contraintes matérielles liées aux serveurs physiques traditionnels.

Ce travaux pratiques (TP) a pour objectif de nous familiariser avec le cycle de vie d'une instance EC2 : de son provisionnement automatisé à sa gestion opérationnelle. Au-delà de la simple mise en service, nous aborderons des problématiques essentielles telles que la sécurité réseau via les groupes de sécurité, la haute disponibilité et la protection contre les erreurs humaines (*Termination* et *Stop Protection*). À travers la mise en œuvre d'un serveur Web sous Amazon Linux 2023, ce TP illustre les principes fondamentaux de l'élasticité et de l'automatisation qui font la force des architectures cloud modernes.

Tâche 1 : Lancement de votre instance Amazon EC2

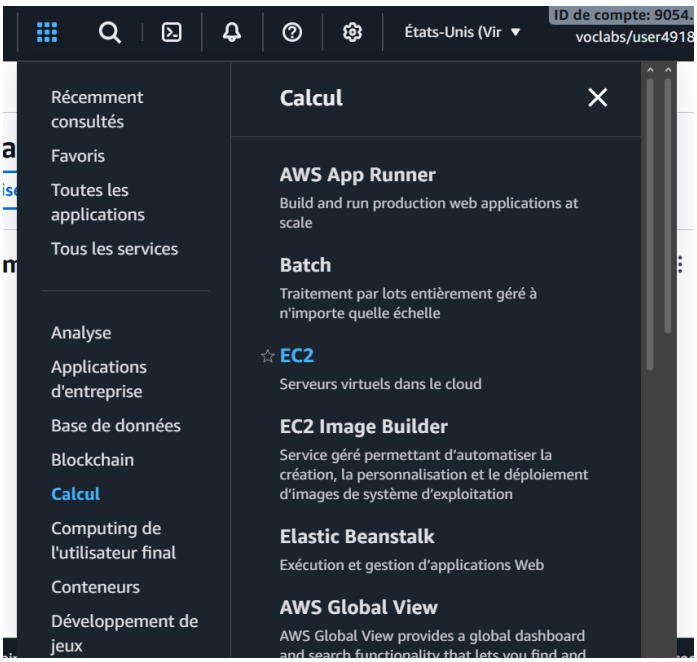


Figure 1 console EC2

L'objectif initial était de déployer une machine virtuelle sous Amazon Linux 2023.

- **Configuration de l'instance :** Nommée Web Server, de type t2.micro (1 vCPU, 1 GiB RAM).
- **Réseau :** Déploiement dans le Lab VPC sur un sous-réseau public.
- **Sécurité initiale :** Création d'un groupe de sécurité nommé Web Server security group sans règles entrantes au départ.
- **User Data (Script d'automatisation) :** Utilisation d'un script Bash pour installer Apache (httpd), démarrer le service et créer une page index.html personnalisée dès le premier démarrage.
- **Protections :** Activation de la **Termination Protection** (Protection contre la suppression accidentelle).

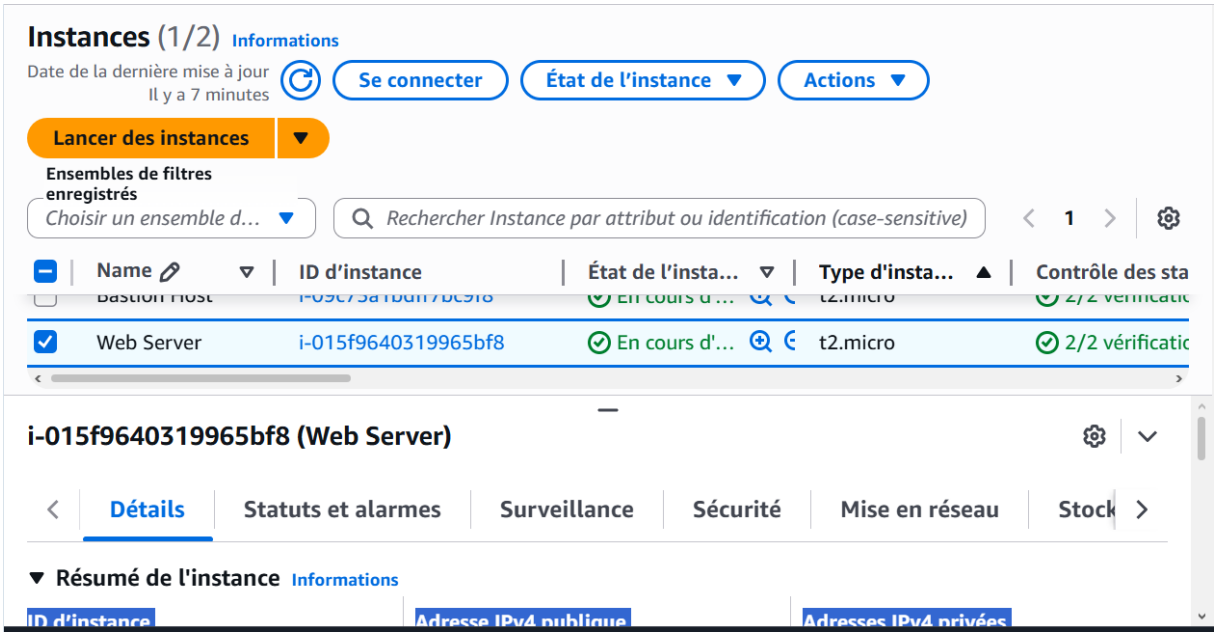


Figure 2 État de l'instance EC2 après le lancement initial.

Tâche 2 : Surveillance de votre instance

La surveillance constitue une part importante de la gestion de la fiabilité, de la disponibilité, ainsi que des performances de vos instances Amazon Elastic Compute Cloud (Amazon EC2) et de vos solutions AWS.

AWS fournit des outils pour vérifier la santé de l'instance sans s'y connecter directement.

- **Status Checks :** Vérification que le matériel AWS et le système d'exploitation de l'instance fonctionnent correctement.
- **Monitoring :** Observation des graphiques CloudWatch (utilisation CPU, trafic réseau).

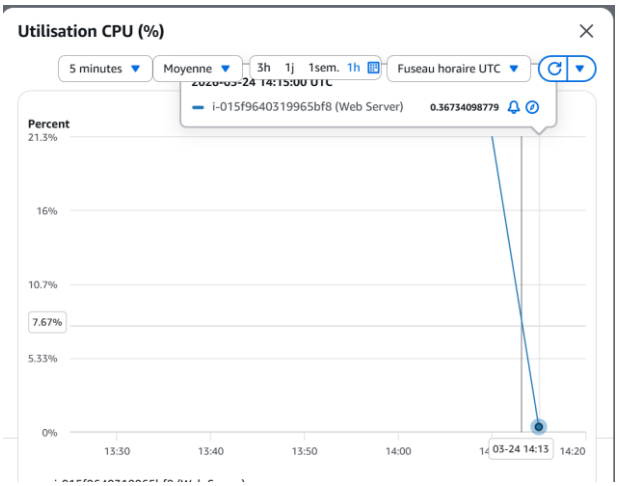


Figure 3 Exemple métriques Amazon CloudWatch

- **System Log :** Analyse des logs de démarrage pour confirmer que le script User Data a bien installé le package `httpd`.

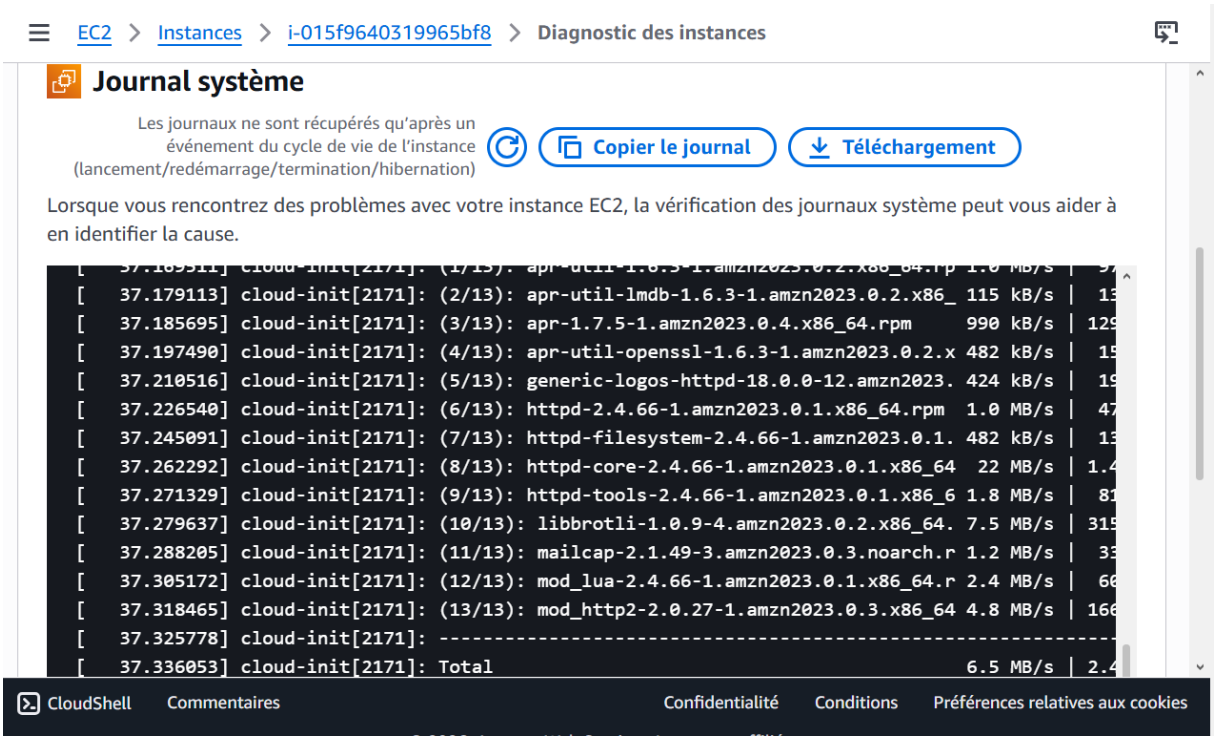


Figure 4 Log système confirmant l'exécution réussie du script de post-installation.

- **Instance Screenshot :** Capture visuelle de la console de l'instance pour confirmer qu'elle est sur l'écran de login.

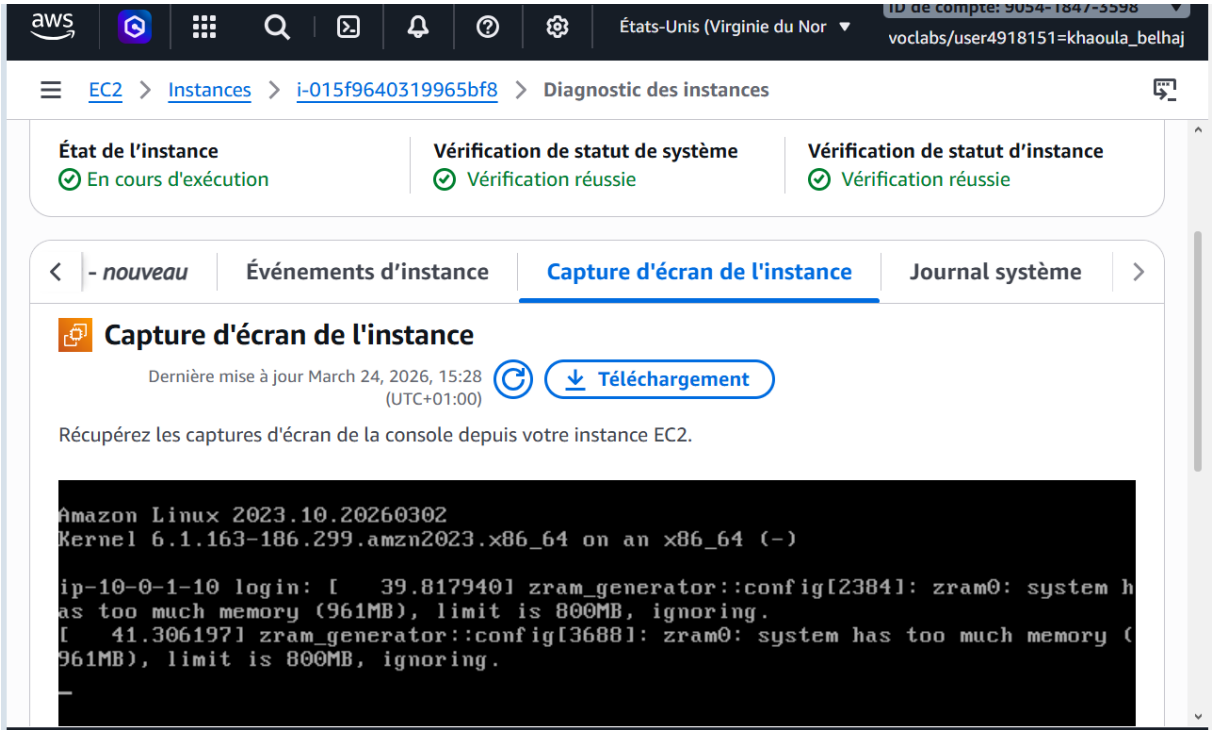


Figure 5 Obtenir la capture de l'instance

Tâche 3 : Mise à jour de votre groupe de sécurité et accès au serveur web

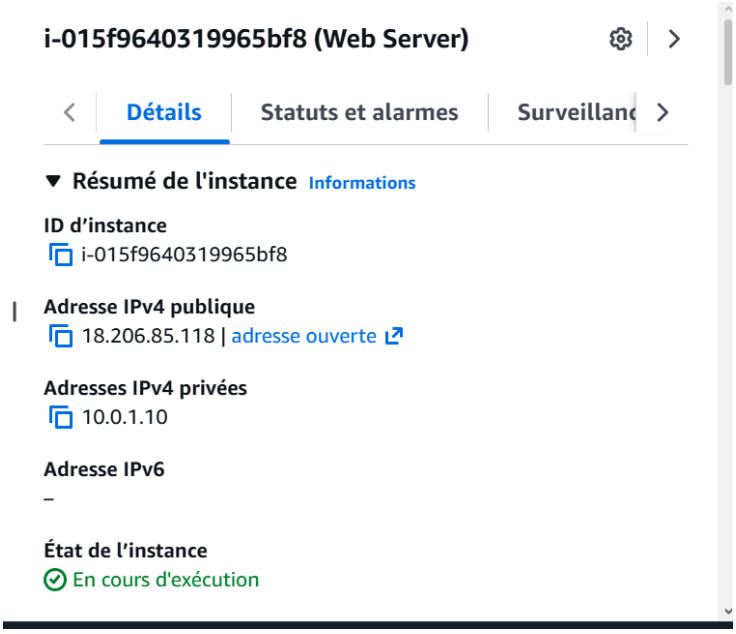


Figure 6 Web Server Détails

Initialement, le serveur était inaccessible via le navigateur car le port 80 était fermé.

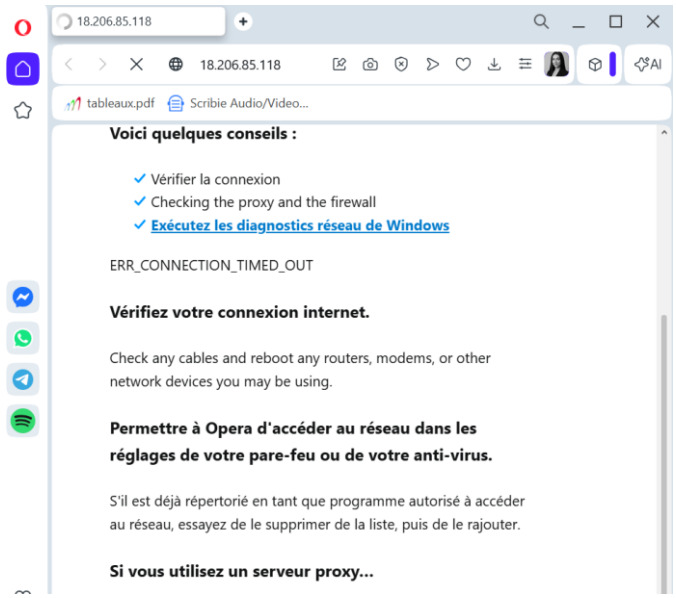


Figure 7 Serveur inaccessible

- **Action :** Modification du groupe de sécurité pour ajouter une règle **Inbound** autorisant le trafic **HTTP** sur le port 80 depuis n'importe quelle adresse (`0.0.0.0/0`).

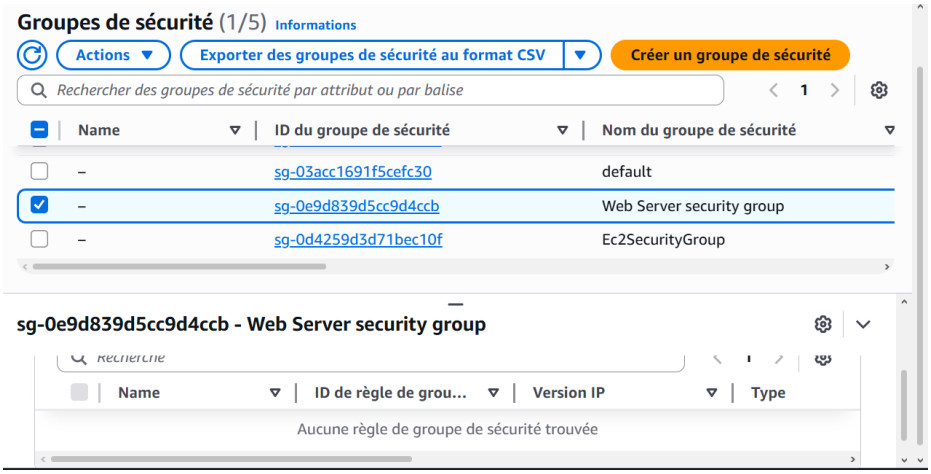


Figure 8 Modification du groupe de sécurité

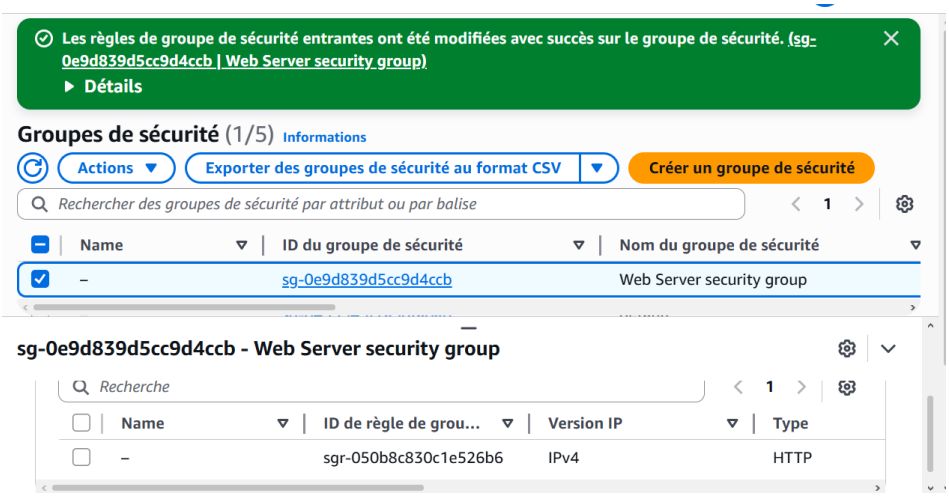


Figure 9 Ajout HTTP

- **Résultat :** L'accès à l'adresse IP publique de l'instance affiche désormais le message : *"Hello From Your Web Server!"*.

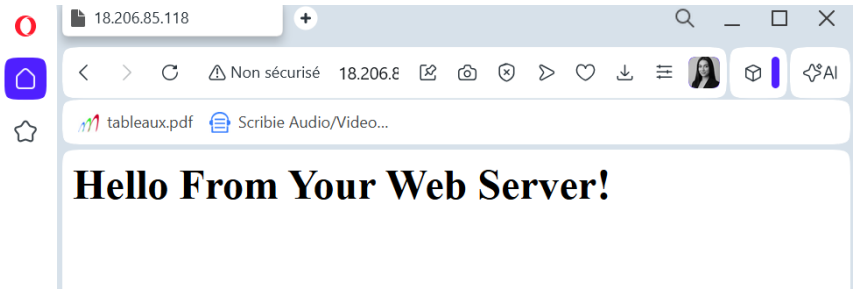


Figure 10 Confirmation de la connectivité HTTP après modification des règles de sécurité

Tâche 4 : Redimensionnement de votre instance - type d'instance et volume EBS

L'agilité du cloud permet de modifier les ressources selon les besoins.

- **Arrêt de l'instance :** Obligatoire pour modifier le type d'instance.

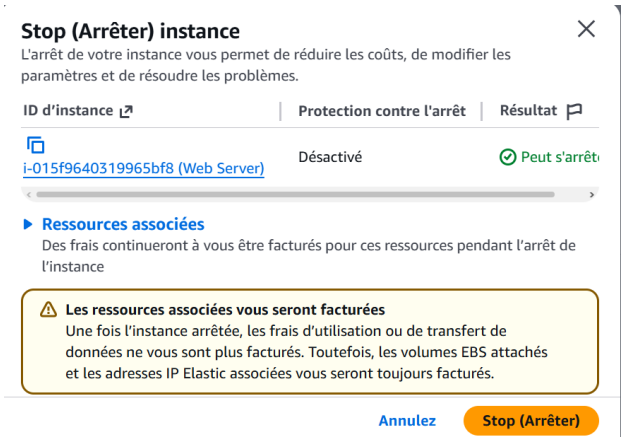


Figure 11 Arrêt de l'instance

- **Changement de type :** Passage de `t2.micro` à `t2.small` (doublement de la RAM).

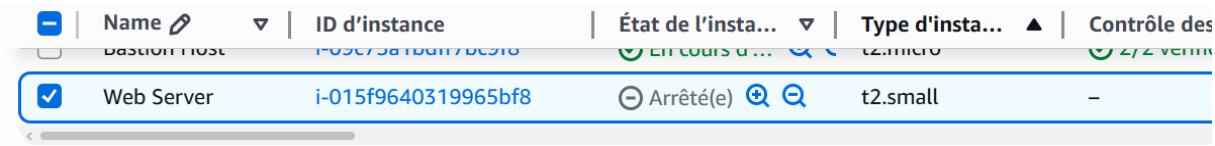


Figure 12 Passage de t2.micro à t2.small

- **Modification du volume EBS :** Augmentation de la taille du disque de **8 GiB à 10 GiB**.

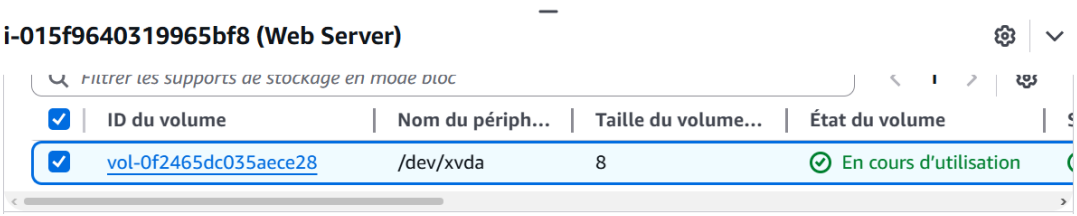


Figure 13 Modification du volume EBS

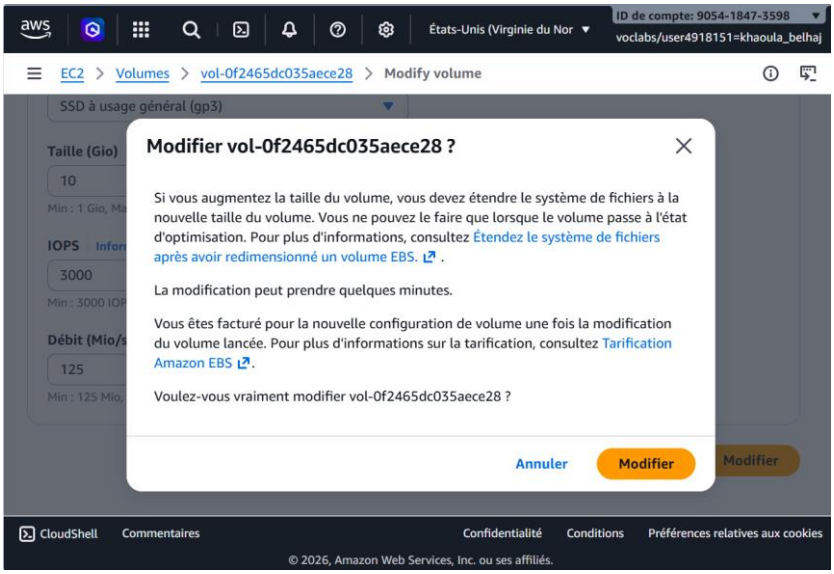


Figure 14 Extension de la capacité de stockage du volume racine.

- **Stop Protection :** Activation de la protection contre l'arrêt pour éviter les interruptions de service involontaires.

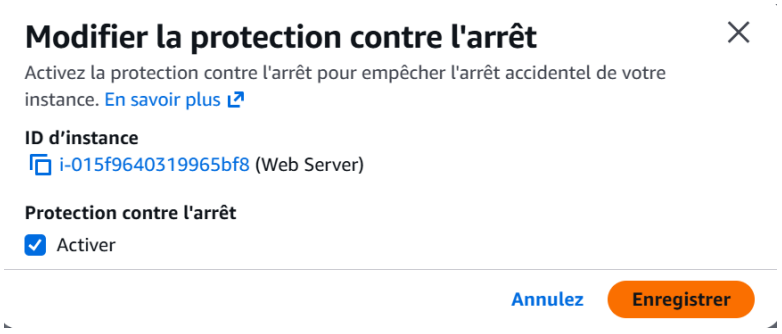


Figure 15 Activation de la protection contre l'arrêt

Tâche 5 : Exploration des limites d'EC2

Amazon EC2 fournit différentes ressources que vous pouvez utiliser. Ces ressources incluent des images, des instances, des volumes et des instantanés (snapshots). Lorsque vous créez un compte AWS, des limites par défaut s'appliquent à ces ressources en fonction de la région.

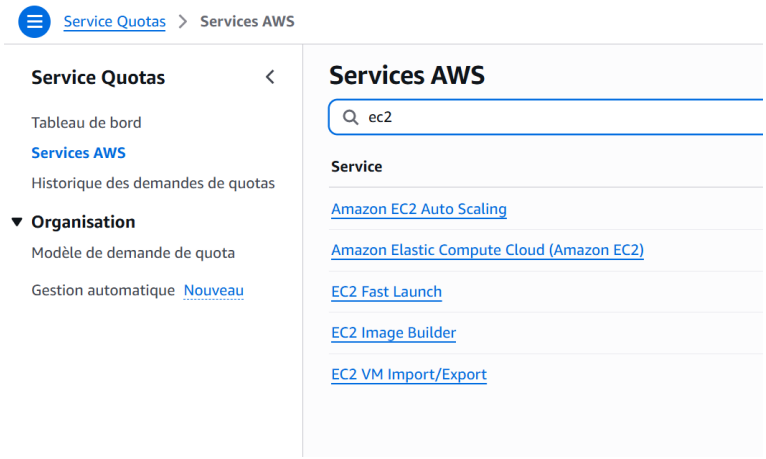


Figure 16 Service Quotas

- **Service Quotas :** Exploration des limites par défaut du compte (ex: nombre maximal d'instances On-Demand par région).

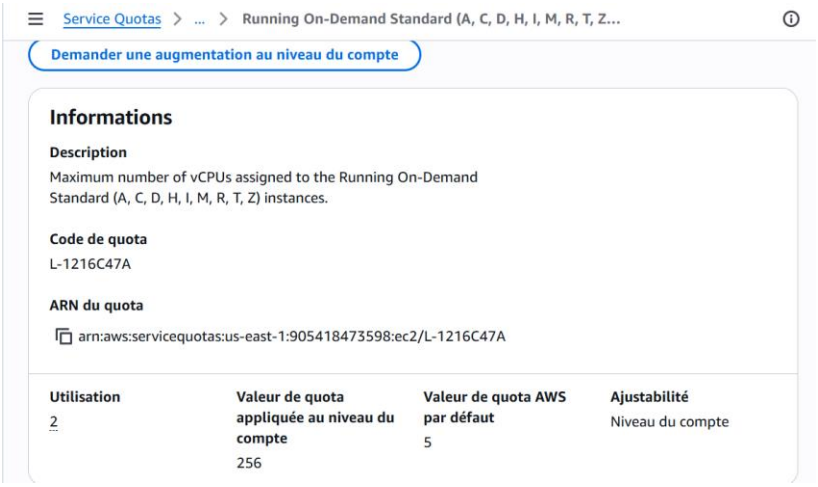


Figure 17 Running On-Demand Standard

Tâche 6 : Test de la protection contre l'arrêt

- **Test de Stop Protection :** Tentative d'arrêt de l'instance qui a échoué avec un message d'erreur, prouvant que la protection est active.

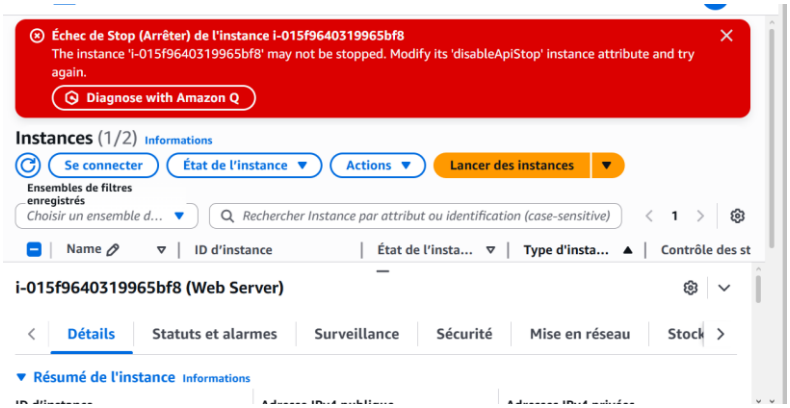


Figure 18 Démonstration de la sécurité empêchant l'arrêt accidentel du serveur

- **Désactivation et Arrêt final :** Désactivation manuelle de la protection pour pouvoir enfin stopper l'instance.

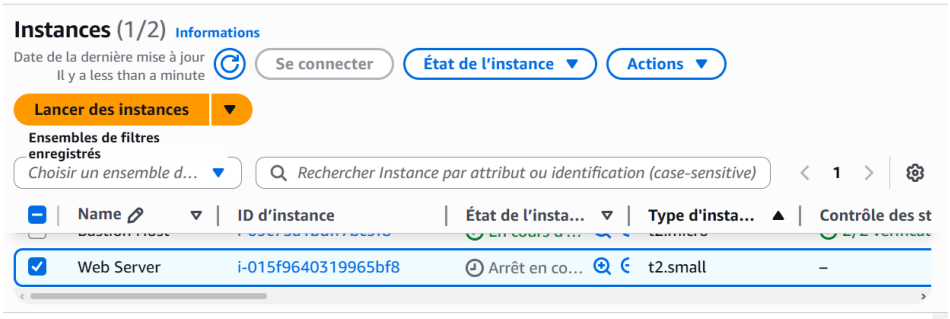


Figure 19 Désactivation et Arrêt final

Fin de l'atelier

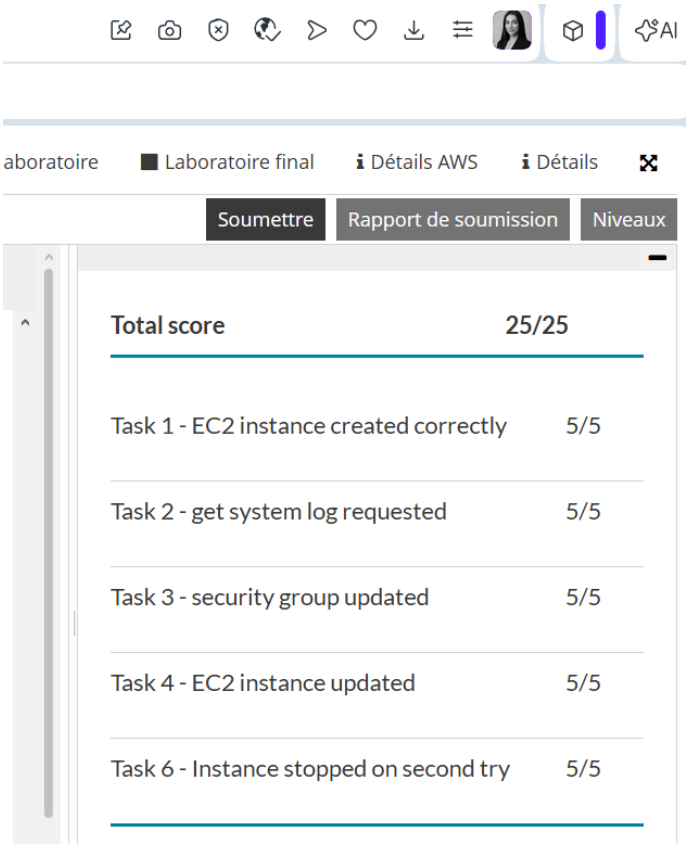


Figure 20 Fin de l'atelier

Conclusion

Synthèse et réflexions sur l'élasticité Cloud

Au terme de ce TP, nous avons pu appréhender la flexibilité qu'offre l'écosystème AWS. L'automatisation du déploiement via des scripts *User Data* a démontré l'efficacité de l'Infrastructure as Code (IaC) pour garantir la cohérence des environnements.

Plusieurs enseignements majeurs peuvent être tirés de cette expérience :

1. **La gestion de la sécurité est granulaire :** Le groupe de sécurité agit comme une première ligne de défense indispensable. Sa configuration, initialement restrictive, souligne l'importance du principe du « moindre privilège » dans toute stratégie réseau.
2. **L'agilité opérationnelle est réelle :** Le passage d'une instance `t2.micro` à une `t2.small` et l'extension du volume EBS confirment la capacité du cloud à s'adapter dynamiquement à une charge applicative variable, sans nécessiter de refonte matérielle.
3. **La résilience par la configuration :** L'activation des protections (Termination et Stop Protection) souligne que la gestion des infrastructures cloud demande une vigilance particulière sur les politiques de contrôle d'accès pour éviter toute interruption accidentelle de service.

En somme, Amazon EC2 ne se limite pas à la fourniture de machines virtuelles ; il offre un cadre complet de gestion, de surveillance (CloudWatch) et d'évolutivité. Ces compétences sont le socle nécessaire pour toute personne souhaitant évoluer vers des architectures cloud plus complexes, telles que le déploiement de clusters auto-scalables ou d'architectures distribuées sur plusieurs zones de disponibilité.

Automatisation avec AWS Lambda et EventBridge

Introduction :

Le concept du "Serverless"

Dans le paysage actuel du Cloud Computing, la gestion des infrastructures ne se limite plus au simple provisionnement de serveurs virtuels. La montée en puissance des architectures **Serverless** (ou "sans serveur") marque une rupture majeure avec les modèles traditionnels. Le concept de "Serverless" ne signifie pas l'absence de serveurs, mais plutôt le fait que l'utilisateur est totalement déchargé de la gestion, du maintien en condition opérationnelle, et de la mise à l'échelle des ressources sous-jacentes.

Le service **AWS Lambda** est au cœur de cette révolution. Il permet d'exécuter du code informatique en réponse à des événements spécifiques, sans avoir à gérer l'infrastructure qui héberge ce code. Ce modèle offre une agilité sans précédent : le code ne consomme des ressources (et ne coûte de l'argent) que lorsqu'il est réellement exécuté.

Ce travaux pratiques (TP) a pour objectif d'illustrer la synergie entre la puissance de calcul éphémère de Lambda et la capacité d'orchestration temporelle d'**Amazon EventBridge**. Nous allons construire une solution d'automatisation, souvent surnommée « Stopinator », capable de réguler l'état des ressources EC2 sans intervention humaine. À travers cette expérimentation, nous aborderons trois piliers fondamentaux de l'administration cloud :

1. **L'automatisation pilotée par les événements :** Comprendre comment un déclencheur (EventBridge) initie une chaîne logique de traitement.
2. **La sécurité granulaire :** Appliquer le principe du moindre privilège en configurant des rôles **IAM** (*Identity and Access Management*) qui limitent les actions de la fonction Lambda au strict nécessaire.
3. **L'interaction via API :** Utiliser le SDK **Boto3** pour permettre à notre code Python d'interagir directement avec l'infrastructure AWS et de manipuler des ressources de haut niveau comme des instances EC2.

Ce TP constitue une base essentielle pour comprendre comment optimiser les coûts opérationnels et fiabiliser les environnements cloud par le biais de l'automatisation.

Configuration de la fonction Lambda

La première étape a consisté à créer la fonction nommée `myStopinator`.

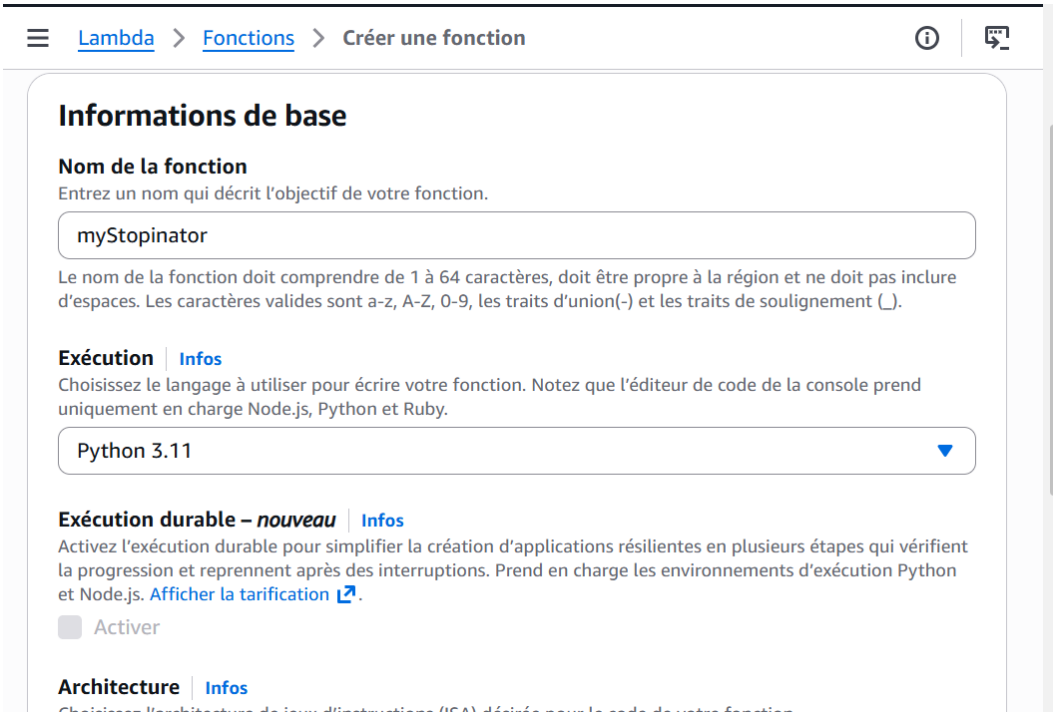


Figure 21 créer la fonction nommée myStopinator

- **Runtime :** Python 3.11.
- **Sécurité (IAM) :** Utilisation du rôle `myStopinatorRole`. Ce point est critique : la fonction Lambda n'a aucun droit par défaut. Ce rôle IAM lui donne la permission spécifique d'exécuter l'action `ec2:StopInstances`.
- **Code source :** Intégration d'un script utilisant la bibliothèque `boto3`. Le script cible une région précise (`us-east-1`) et un ID d'instance spécifique pour lui envoyer l'ordre d'arrêt.

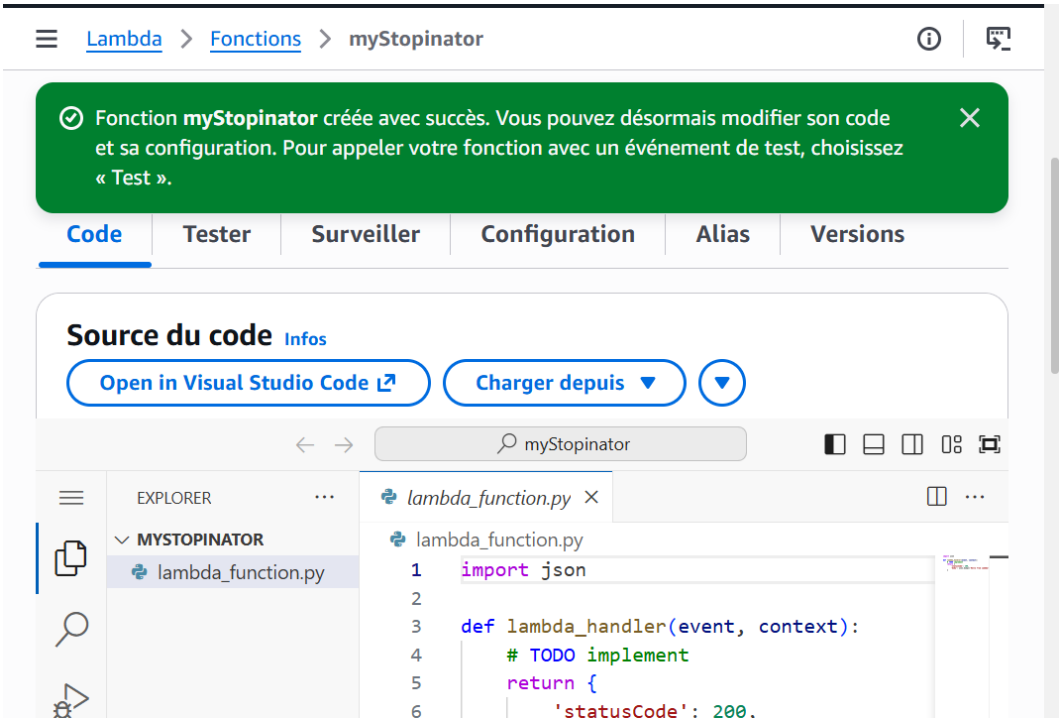


Figure 22 Création de la fonction Lambda avec les permissions d'exécution nécessaires

Tâche 1 : Création d'une fonction Lambda

Pour que la fonction s'exécute sans intervention humaine, nous avons configuré **Amazon EventBridge**.

- **Règle :** `everyMinute`.
- **Expression de planification :** `rate(1 minute)`.

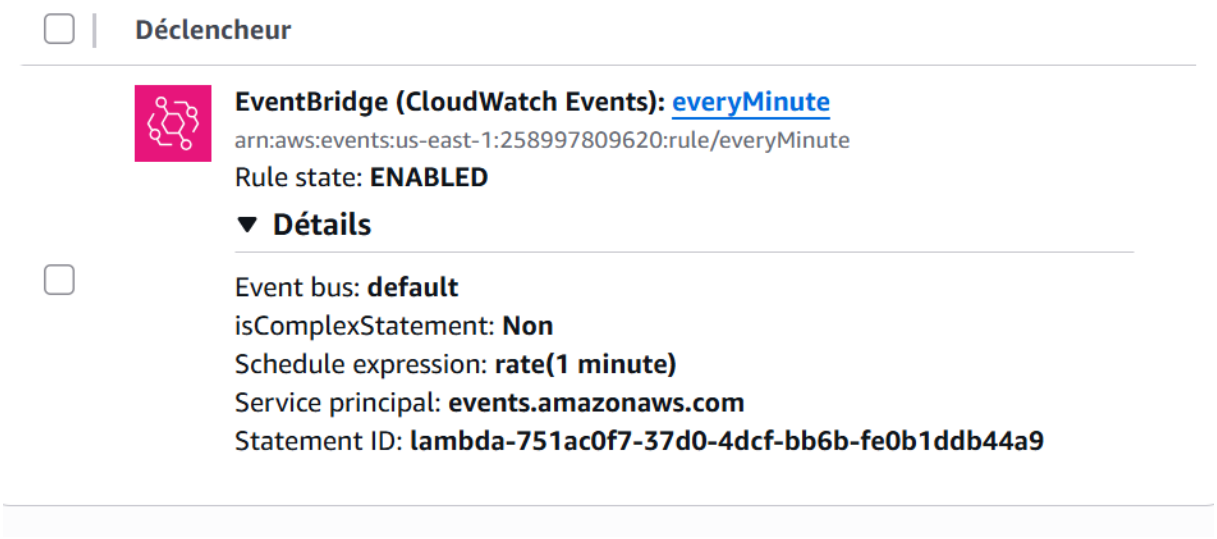


Figure 23 EventBridge

- **Analyse :** Bien que dans un cas réel (production) on utiliserait une expression **Cron** (ex: "arrêter tous les soirs à 20h"), le mode "rate" nous a permis de vérifier instantanément le bon fonctionnement de l'automatisation.

Tâche 2 : Configuration du déclencheur

Une fois la fonction déployée et le déclencheur actif, nous avons observé le comportement de l'instance EC2 nommée `instance1`.

- **Observation :** L'instance passe de l'état `Running` à `Stopping`, puis `Stopped` de manière autonome.
- **Test de résilience :** Même en tentant de redémarrer l'instance manuellement, celle-ci est de nouveau arrêtée par la fonction Lambda dans la minute qui suit.
- **Monitoring :** L'onglet "Monitor" de Lambda confirme les invocations réussies via les logs CloudWatch.

[Image 2 : Graphique de surveillance dans l'onglet Monitor montrant les pics d'invocation (Invocations) chaque minute] > *Légende : Suivi des métriques CloudWatch confirmant l'exécution périodique de la fonction.*

Tâche 3 : Configuration de la fonction Lambda

Le code utilisé est minimaliste mais puissant :

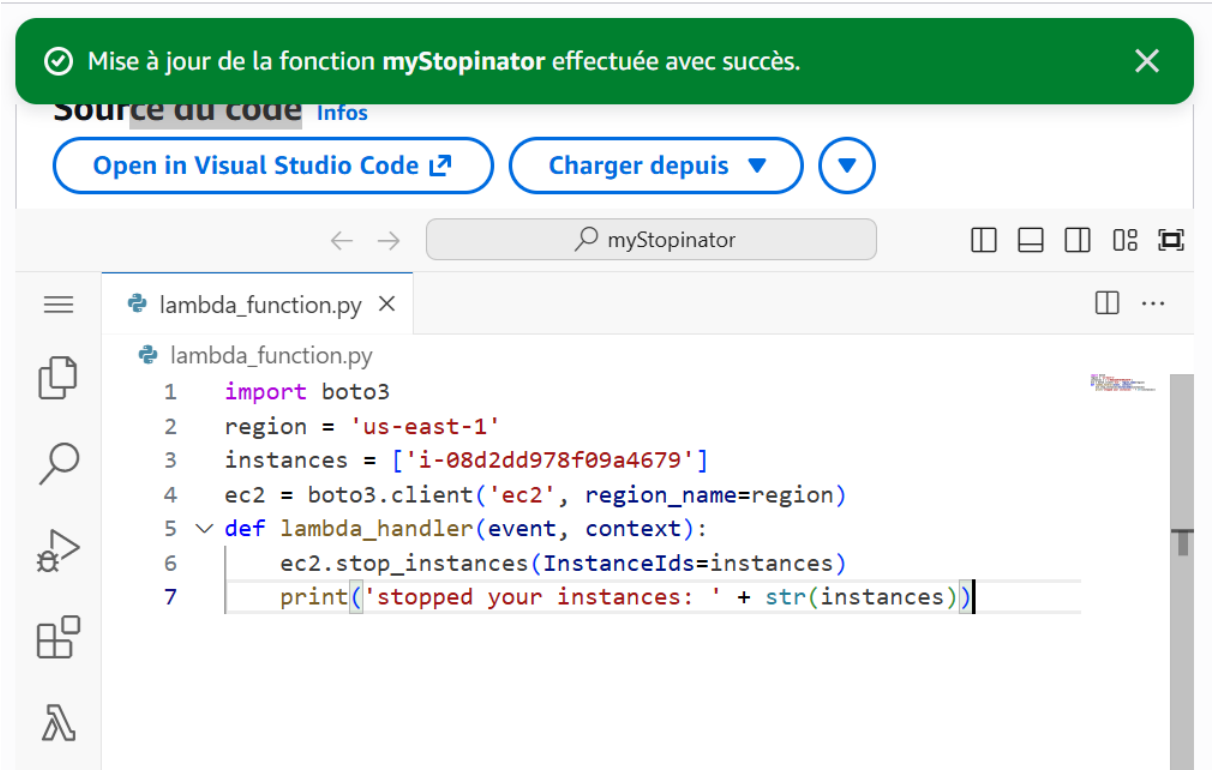


Figure 24 Console AWS Lambda

Ici, `lambda_handler` est le point d'entrée. Dès qu'EventBridge envoie un événement, Lambda exécute cette fonction, qui utilise le client EC2 de Boto3 pour communiquer avec l'API AWS.

Tâche 4 : Vérification du bon fonctionnement de la fonction Lambda

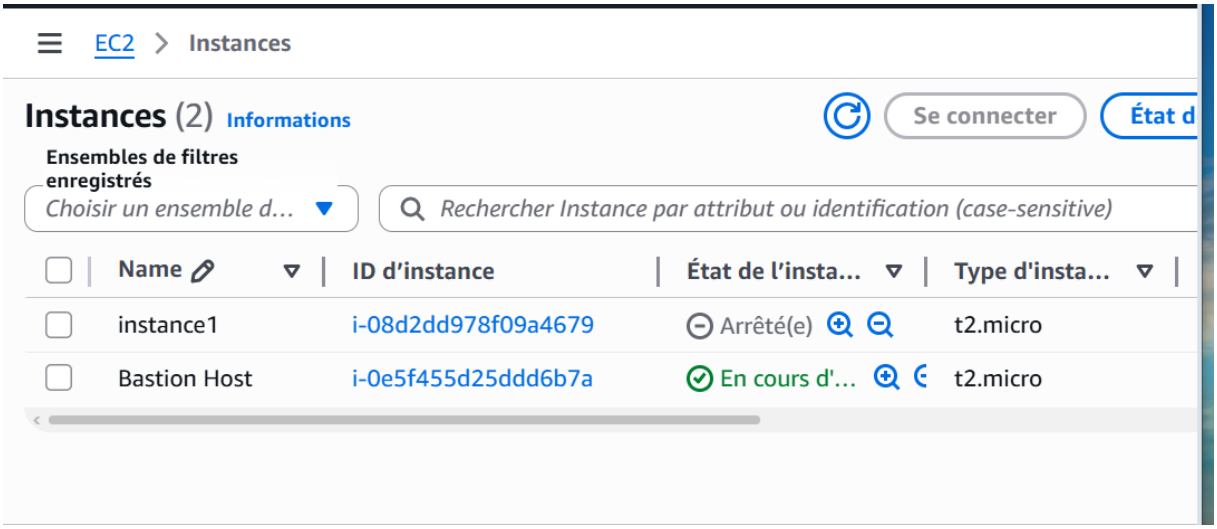


Figure 25 instance arrêtée

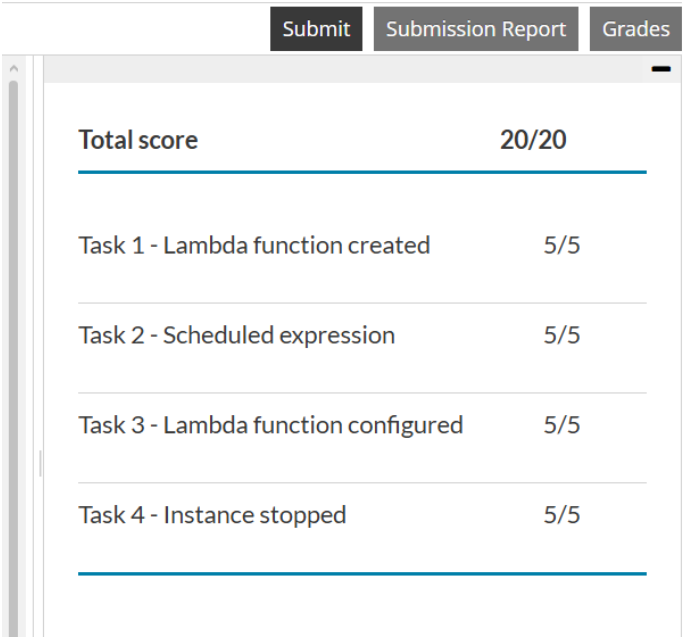


Figure 26 Fin de l'atelier

Conclusion

Ce TP démontre la puissance de l'automatisation Cloud. Nous avons créé un système capable de surveiller et de modifier l'état d'une infrastructure de manière totalement autonome.

Points clés retenus :

1. **Économie de coûts :** Le "Stopinator" est un exemple concret d'outil permettant de réduire la facture Cloud en s'assurant que les instances de test sont arrêtées hors des heures de travail.
2. **Sécurité IAM :** La réussite de la fonction dépend entièrement des permissions accordées au rôle d'exécution.
3. **Scalabilité :** Ce modèle peut être étendu pour arrêter des centaines d'instances simultanément en modifiant simplement la liste des IDs dans le code.